

DZIEŃ 4 — DICTIONARY, ROZBUDOWANE STATE MACHINE, REFRAMEWORK

Moduł 1 — Dictionary

CZYM JEST DICTIONARY?

Słownik (Dictionary) to kolekcja par **klucz-wartość**. Każdy klucz jest unikalny — nie mogą istnieć dwa elementy z tym samym kluczem. Najprościej myśleć o nim jak o encyklopedii: szukasz hasła (klucz) i dostajesz definicję (wartość).

Przypomnienie z Dnia 1-2: Na Dniu 1 omówiliśmy podstawy kolekcji (Array, List, Dictionary). Teraz pokazemy praktyczne zastosowanie Dictionary w realnym scenariuszu — mapowanie kursów walut.

W UiPath najczęściej spotykane typy:

- `Dictionary(Of String, String)` — klucz i wartość to tekst (np. kod waluty → kurs)
- `Dictionary(Of String, Object)` — klucz to tekst, wartość to dowolny typ (tak działa Config w REFramework)
- `Dictionary(Of String, Int32)` — klucz to tekst, wartość to liczba (np. licznik kategorii)

DICTIONARY VS DATATABLE — KIEDY CO WYBRAĆ?

Kryterium	Dictionary	DataTable
Struktura	Klucz → Wartość (1:1)	Wiersze x Kolumny (tabela)
Wyszukiwanie	Błyskawiczne po kluczu ($O(1)$)	Wymaga iteracji po wierszach ($O(n)$) lub LINQ
Kiedy użyć	Mapowanie (kod → nazwa), konfiguracja, cache	Dane tabelaryczne, raporty, import/eksport Excel
Przykład	Kod waluty → kurs	Lista transakcji z wieloma kolumnami
Sortowanie	Brak gwarancji kolejności	Zachowuje kolejność wierszy

ZŁOŻONOŚĆ OBLICZENIOWA — DLACZEGO DICTIONARY JEST SZYBSZY?

Gdy masz 10 000 walut w DataTable i szukasz kursu EUR, musisz przejść potencjalnie przez wszystkie 10 000 wierszy (w najgorszym przypadku). To zajmuje czas proporcjonalny do rozmiaru tabeli.

Dictionary używa techniki zwanej "haszowaniem" — zamienia klucz na liczbę i od razu wie, w którym "schowku" szukać. Niezależnie od tego czy masz 100 czy 10 000 elementów, czas wyszukiwania jest stały (**O(1)**).

W praktyce: jeśli musisz wielokrotnie wyszukiwać wartość po kluczu (np. kurs waluty, tłumaczenie kodu na nazwę), Dictionary będzie **znacznie szybszy** niż iterowanie po DataTable.

INICJALIZACJA

```
' W Assign:  
dict_waluty = New Dictionary(Of String, String)
```

Ważne: Oprócz stworzenia zmiennej w Data Manager i ustawienia typu, trzeba jeszcze napisać komendę `New Dictionary(Of String, String)` w Assign — inaczej zmienna będzie `Nothing` i każda operacja rzuci `NullReferenceException` !

Operacje na Dictionary

Operacja	Wyrażenie VB.NET	Opis
Dodaj/Ustaw wartość	<code>dict("klucz") = "wartosc"</code>	Dodaje lub nadpisuje
Pobierz wartość	<code>dict("klucz")</code>	Rzuca wyjątek jeśli klucz nie istnieje
Sprawdź klucz	<code>dict.ContainsKey("klucz")</code>	Zwraca Boolean
Liczba elementów	<code>dict.Count</code>	Int32
Usun element	<code>dict.Remove("klucz")</code>	Zwraca Boolean

TYPOWE PULAPKI PRZY PRACY Z DICTIONARY

1. KeyNotFoundException

Proba pobrania wartości kluczem, który nie istnieje:

```
' ZLE – rzuci wyjątek jeśli "GBP" nie istnieje:
str_kurs = dict_waluty("GBP")

' DOBRE – najpierw sprawdź:
If dict_waluty.ContainsKey("GBP") Then
    str_kurs = dict_waluty("GBP")
End If
```

2. Case-sensitivity kluczy

Domyslnie Dictionary rozróżnia wielkość liter:

```
dict_waluty("EUR") = "4.55"
dict_waluty("eur") ' → KeyNotFoundException! "eur" ≠ "EUR"
```

Rozwiązanie: przy dodawaniu i pobieraniu zawsze stosuj `.ToUpper` lub `.ToLower` na kluczu.

3. NullReferenceException

Dictionary nie zostało zainicjalizowane:

```
' Stworzenie zmiennej w Data Manager to za mało!
' Trzeba jeszcze:
dict_waluty = New Dictionary(Of String, String)
```

Cwiczenia — Moduł 1

Cwiczenie — Kursy walut NBP → Dictionary

Cel: Otwórz stronę NBP z kursami walut, pobierz tabelę (Table Extraction), zapisz do Excela, przypisz do Dictionary i wyświetl kurs Euro.

CZEGO SIE NAUCZYSZ

`Extract Table Data` — scrapowanie tabel ze stron www

Inicjalizacja Dictionary — MUSI być `New Dictionary(Of String, String)` w Assign

`dict(klucz) = wartosc` — jednocześnie dodaje i nadpisuje

Dostęp do wartości: `dict_waluty("1 EUR").ToString`

Klasyczny pattern mapowania danych do Dictionary: jedna kolumna jako klucze, druga jako wartości

Modul 2 — Rozbudowane State Machine

BUSINESSRULEEXCEPTION VS SYSTEMEXCEPTION — KLUCZOWE ROZROZNIENIE

W automatyzacji procesów wyznaczamy dwa fundamentalnie różne typy błędów:

Cecha	BusinessRuleException (BE)	System Exception (SE)
Definicja	Błąd wynikający z danych lub reguł biznesowych	Błąd techniczny/systemowy
Przykłady	Brak NIPu w formularzu, kwota ujemna, niepoprawny email, dane nienumeryczne	Strona się nie załadowała, aplikacja przestała odpowiadać, timeout, brak internetu
Czy wina robota?	Nie — dane wejściowe są złe	Nie — środowisko jest niestabilne
Czy ponawiać?	NIE — ponawianie z tymi samymi złymi danymi da ten sam wynik	TAK — ponowna próba może się udać (np. strona się załaduje)
Reakcja w SM	Pomin transakcję (Failed + Business) → następna	Restart — zamknij aplikację, otwórz ponownie, spróbuj jeszcze raz
W UiPath	<code>Throw New BusinessException("opis")</code>	Nieobsłużony Exception (np. SelectorNotFoundException, TimeoutException)

Dlaczego to rozróżnienie jest ważne?

Wyobraź sobie, że robot przetwarza faktury. Faktura #123 ma literówkę w NIPie (BE) — ponawianie 10 razy nic nie da. Ale jeśli strona podatki.gov.pl nie odpowiada (SE) — po 30 sekundach może zadziałać.

W REFramework to rozróżnienie jest wbudowane:

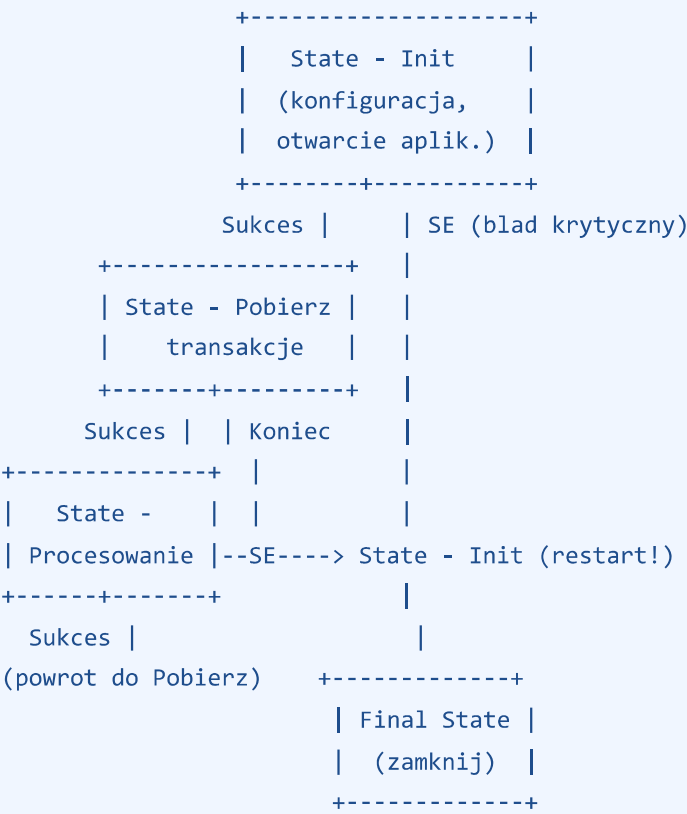
BE → transakcja oznaczona jako Failed (Business), robot przechodzi do następnej

SE → robot restartuje aplikację (wraca do stanu Init) i ponawia próbę

STANY PROCESU W ROZBUDOWANEJ STATE MACHINE

Stan	Opis	Transitions
State - Init	Inicjalizacja: zbierz config, otwórz aplikację, pobierz dane	Sukces → Pobierz transakcje; SE → Final State
State - Pobierz jedną transakcję	Sprawdź czy są dane, pobierz wiersz z DataTable	Sukces → Procesowanie; Koniec → Final State
State - Procesowanie jednej transakcji	Wpisz dane do Robotivity, obsłuż Business/System Exception	Sukces → Pobierz transakcje; SE → Init (restart!)
Final State	Loguj status, zamknij aplikację	— (koniec)

Diagram przeplywu stanow



KLUCZOWE ZMIENNE

Nazwa	Typ	Opis
TransactionNumber	Int32 (default=1)	Numer biezacego wiersza
dict_Config	Dictionary(Of String, String)	Ustawienia (FilePath, SheetName)
dt_transakcje	DataTable	Dane z Transactions.xlsx
row_TransactionItem	DataRow	Biezacy wiersz (transakcja)
SystemException	Exception	Wyjatek systemowy (jesli wystapi)
BusinessRuleException	BusinessRuleException	Wyjatek biznesowy

LOGIKA POSZCZEGOLNYCH STANOW

State - Init

1. If — dict_Config is Nothing → pierwsze uruchomienie:
- Zbierz ustawienia do Dictionary (FilePath, SheetName)
 - Kill Process excel

- Read Range → `dt_transakcje`
- 2. Kill Process `robotivity` (wyczyszc srodowisko)
- 3. Otworz Robotivity + zaloguj sie (z Retry Scope!)
- 4. **Try Catch** wokol wszystkiego — jesli Exception → `SystemException = exception`
- 5. Transitions:
 - `SystemException IsNot Nothing` → Final State (blad krytyczny)
 - Else → State - Pobierz transakcje

State - Pobierz jedna transakcje

1. Sprawdz `TransactionNumber <= dt_transakcje.Rows.Count`
2. Jesli tak → `row_TransactionItem = dt_transakcje.Rows(TransactionNumber - 1)`
3. Jesli nie → `row_TransactionItem = Nothing`
4. Transitions:
 - `row_TransactionItem Is Nothing` ("Koniec") → Final State
 - Else ("Sukces") → State - Procesowanie

State - Procesowanie

1. Wyczyszc `BusinessRuleException = Nothing` (wazne, aby nie wejsc w nieskonczona petle!)
2. Wyciagnij CashIn, OnUsCheck, NotOnUsCheck z `row_TransactionItem`
3. Sprawdz `.IsNumeric` — jesli nie → `BusinessRuleException = New BusinessRuleException("Dane nie sa numeryczne")`
4. Jesli numeryczne → Type Into w Robotivity + Click Accept
5. `TransactionNumber = TransactionNumber + 1`
6. Transitions:
 - `SystemException IsNot Nothing` ("SE") → State - Init (restart!)
 - Else ("Sukces") → State - Pobierz transakcje

Final State

- If `SystemException Is Nothing` → "Robot wykonal prace" (Info)
- Else → "Blad pracy robota: " + `SystemException.Message` (Error)
- Kill Process `robotivity`

Cwiczenia — Moduł 2

Cwiczenie — Rozbudowane State Machine

Cel: Zbuduj State Machine z pełną obsługą wyjątków — wzorec zbliżony do REFramework. Robot przetwarza transakcje z Excela w Robotivity, rozróżniając błędy biznesowe (dane nienumeryczne) od systemowych (awaria aplikacji).

CZEGO SIE NAUCZYSZ

Dictionary jako Config — wzorec REFramework (Key-Value settings)

`If dict_Config is Nothing` — inicjalizacja tylko raz (przy restarcie nie powtarza)

Retry Scope — automatyczne ponawianie logowania (2 próby, 3s odstęp)

BusinessRuleException vs SystemException — różne typy błędów, różne reakcje

Czyszczenie BusinessRuleException na początku procesowania — zapobiega nieskończonej petli

Transition "SE" z Procesowania → Init — restart aplikacji po błędzie systemowym

TransactionNumber — ręczny licznik wierszy (zamiast Get Transaction Item z Orchestratora)

Ważne: To jest bezpośredni wstęp do REFramework! Zauważ analogie: Init ↔ InitAllApplications, Pobierz transakcje ↔ GetTransactionData, Procesowanie ↔ Process, Final State ↔ CloseAllApplications.

Moduł 3 — REFramework: wprowadzenie

DLACZEGO REFRAMEWORK? JAKIE PROBLEMY ROZWIĄDUJE?

Wyobraź sobie, że piszysz robota, który przetwarza 500 faktur dziennie. Bez frameworka musisz sam zaprogramować:

- Co się stanie, gdy strona się zawiesi w trakcie przetwarzania faktury #237?
- Jak zrestartować aplikację i wrócić do przetwarzania?
- Ile razy ponawiać próbę przed oznaczeniem transakcji jako "Failed"?
- Gdzie trzymać konfigurację (URL, hasła, timeouty)?
- Jak logować postęp i błędy?

Każdy programista rozwiąże to inaczej — co prowadzi do chaosu w utrzymaniu. REFramework standaryzuje te rozwiązania.

CO REFRAMEWORK ZAPEWNI "OUT OF THE BOX"

- 1. **Automatyczny retry** — po System Exception robot restartuje aplikacje i probuje ponownie (konfigurowalne MaxRetryNumber)
- 2. **Rozroznienie bledow** — Business Exception (pomin transakcje) vs System Exception (restart)
- 3. **Centralna konfiguracja** — Config.xlsx zamiast hardkodowanych wartosci
- 4. **Logging** — automatyczne logowanie statusow transakcji
- 5. **Screenshots przy bledach** — TakeScreenshot.xaml robi zrzut ekranu przy wyjatku
- 6. **Separacja kodu** — Framework (nie ruszaj) vs Process.xaml (twoja logika)

STRUKTURA FOLDEROW REFRAMEWORK

Folder/Plik	Opis
Main.xaml	Glowny State Machine (Init → Get Transaction → Process → End)
Framework/InitAllSettings.xaml	Wczytuje Config.xlsx do Dictionary
Framework/InitAllApplications.xaml	Otwiera potrzebne aplikacje
Framework/GetTransactionData.xaml	Pobiera kolejny element (z Queue lub innego zrodla)
Framework/Process.xaml	Logika biznesowa — przetwarzanie jednej transakcji
Framework/SetTransactionStatus.xaml	Ustawia status transakcji (Successful/Failed)
Framework/CloseAllApplications.xaml	Zamyka aplikacje (graceful close)
Framework/KillAllProcesses.xaml	Wymusza zamkniecie procesow (force kill)
Framework/RetryCurrentTransaction.xaml	Logika ponawiania transakcji
Framework/TakeScreenshot.xaml	Screenshot przy bledzie
Data/Config.xlsx	Plik konfiguracyjny (Settings, Constants, Assets)
Data/Input/	Dane wejscowe
Data/Output/	Dane wyjscowe
Data/Temp/	Pliki tymczasowe

STANY REFRAMEWORK (MAIN.XAML)

Stan	Co robi	Przejścia
Init	InitAllSettings + InitAllApplications	Sukces → Get Transaction; SE → End
Get Transaction Data	Pobiera kolejny element do przetworzenia	Jest element → Process; Brak → End
Process Transaction	Wykonuje Process.xml + SetTransactionStatus	Sukces → Get Transaction; BE → Get Transaction; SE → Init
End Process	CloseAllApplications	— (koniec)

Cykl życia transakcji w REFramework

```

[Init] → ustawienia zaladowane → [Get Transaction]
      |
      | Jest transakcja? → NIE → [End Process]
      | TAK
      | [Process Transaction]
      |   |   |   |
      | Sukces  BE  SE
      |   |   |   |
      | [Get   [Get  [Init] ← restart!
      Trans.] Trans.] (retry)

```

- **Sukces** → transakcja oznaczona jako Successful, pobierz następna
- **Business Exception** → transakcja oznaczona jako Failed (Business), pobierz następna (NIE ponawiaj)
- **System Exception** → restart aplikacji, ponow te sama transakcje (do MaxRetryNumber razy)

CONFIG.XLSX — TRZY ARKUSZE

Arkusz	Opis	Kiedy uzywac	Przyklad
Settings	Ustawienia procesu, ktore moga sie zmieniac	Nazwy kolejek, URL-e, konfiguracja retry	OrchestratorQueueName = "MojaKolejka", MaxRetryNumber = 3
Constants	Stale wartosci, ktore sie nie zmieniaja	Timeouty, formaty dat, nazwy arkuszy	Timeout_General = 30000, DateFormat = "dd-MM-yyyy"
Assets	Nazwy assetow z Orchestratora (nie same wartosci!)	Nazwy credentiali, sciezki do aplikacji	CredentialName = "Robot_Login", AppPath_Asset = "Robotivity_Path"

Wazna roznica: W arkuszu Assets nie wpisujemy samych hasel ani wartosci — tylko **nazwy assetow** z Orchestratora. InitAllSettings wczytuje te nazwy, a potem robot uzywa Get Asset / Get Credential z tymi nazwami, aby pobrac rzeczywiste wartosci.

Uwaga techniczna: Config w REFramework jest typu `Dictionary(Of String, Object)` (nie `String, String`!) — dlatego przy odczycie zawsze potrzebny jest `.ToString`: `Config("OrchestratorQueueName").ToString`

ARGUMENTY MAIN.XAML

- `in_OrchestratorQueueName` — nazwa kolejki (mozna nadpisac z Orchestratora)
- `in_OrchestratorQueueFolder` — folder Orchestratora

Co edytuje programista w REFramework?

W praktyce programista edytuje glownie:

- `Process.xaml` — logika biznesowa (przetwarzanie jednej transakcji)
- `InitAllApplications.xaml` — otwieranie aplikacji i logowanie
- `Config.xlsx` — ustawienia, stale, nazwy assetow
- `CloseAllApplications.xaml` — zamykanie aplikacji

Reszta plikow Framework to boilerplate, ktory dziala "out of the box".

ALTERNATYWA BEZ ORCHESTRATORA — WINDOWS CREDENTIAL MANAGER

Jesli nie masz dostepu do Orchestratora, mozesz uzyc **Windows Credential Manager** jako alternatywy:

1. Panel sterowania → Menedzer poswiadczen → Poswiadczenia systemu Windows → Dodaj
2. W UiPath: **Get Secure Credential** zamiast **Get Credential**
 - Target: nazwa poswiadczenia z Credential Manager
 - Output: Username (String), Password (SecureString)

Wazne: Windows Credential Manager dziala tylko lokalnie na danej maszynie. W produkcji zawsze uzywamy Orchestrator Assets.

Modul 4a — Dispatcher

Cwiczenie — Dispatcher do kolejki

Cel: Załaduj transakcje z Transactions.xlsx do kolejki Orchestratora (hurtowo).

CZEGO SIE NAUCZYSZ

`Kill Process "excel"` — zapobiega blokadzie pliku przez otwarty Excel

`Bulk Add Queue Items` — szybki sposób na załadowanie wielu elementów z DataTable

`FolderPath` — ścieżka folderu w Orchestratorze (My Workspace)

Dispatcher jest prosty — cała złożoność jest po stronie Performera

Modul 4b — REFramework Performer

Cwiczenie — Performer na bazie REFramework

Cel: Używając szablonu REFramework, stwórz Performera, który pobiera transakcje z kolejki i przetwarza je w Robotivity.

CO NALEŻY SKONFIGUROWAĆ

1. Config.xlsx

- Settings: `OrchestratorQueueName` = nazwa kolejki
- Settings: `OrchestratorQueueFolder` = folder Orchestratora
- Assets: nazwy assetów (credentials, ścieżka do aplikacji)

2. InitAllApplications.xaml

- Get Credential → Username, Password
- Get Asset → ścieżka do Robotivity
- Otwórz Robotivity i zaloguj się

3. GetTransactionData.xaml

- Domyślnie: Get Transaction Item z kolejki (już skonfigurowane w template)

4. Process.xaml

- Wyciągnij dane z `in_TransactionItem.SpecificContent`
- Sprawdź `.IsNumeric`
- Jeśli numeryczne → Type Into w Robotivity + Click Accept
- Jeśli nie → `Throw New BusinessException("Dane nie są numeryczne")`

5. CloseAllApplications.xaml

- Kill Process `robotivity`

CZEGO SIĘ NAUCZYSZ

REFramework automatycznie obsługuje: retry, logging, status transakcji, restart po SE

Programista skupia się na: `Process.xaml` (logika) + `InitAllApplications.xaml` (setup)

`Throw New BusinessException(...)` — sygnalizuje błąd biznesowy (transakcja = Failed)

Nieobsłużony Exception w Process.xaml → REFramework traktuje jako System Exception → restart Init

Config.xlsx centralizuje wszystkie ustawienia — łatwa zmiana bez edycji workflow

Wskazówka: REFramework to standard produkcyjny. Większość pracy to edycja Process.xaml i Config.xlsx.

Przydatne

DEBUGOWANIE — DZIEŃ 3 I 4

Flowchart i State Machine

- **Breakpoint w Flowchart:** Kliknij prawym na FlowStep → Toggle Breakpoint. Robot zatrzyma się przed wykonaniem tego kroku.
- **Sledzenie Transitions w State Machine:** W trybie Debug (F5) UiPath podświetla aktualny State i Transition.
- **Locals Panel:** W trybie Debug panel Locals pokazuje wartości wszystkich zmiennych — przydatne do sprawdzenia `TransactionItem`, `SystemException` itp.
- **Slow Step:** Włącz Slow Step w toolbar — spowalnia wykonanie, widoczne każde przejście między stanami.

TYPOWE PROBLEMY I ROZWIĄZANIA

Problem	Przyczyna	Rozwiązanie
"Variable not found"	Scope zmiennej za mały	Przesun deklaracje zmiennej wyżej (np. na poziom StateMachine)
NullReferenceException w Dictionary	Dictionary nie zainicjalizowane	Dodaj <code>New Dictionary(Of String, String)</code> w Assign
Queue Item pusty	Brak danych w kolejce	Najpierw uruchom Dispatcher
"Queue not found"	Zła nazwa kolejki lub folderu	Sprawdź QueueName i FolderPath w Orchestratorze
Nieskonczona petla w State Machine	Brak warunku wyjścia	Dodaj czyszczenie wyjątków + poprawne Transitions
Hasło widoczne w logach	Użycie Text zamiast SecureText	W Type Into użyj SecureText dla hasła
Plik zablokowany (Excel)	Excel otwarty w tle	Kill Process <code>excel</code> przed Read Range
Path Exists zwraca False	Zła ścieżka lub PathType	Sprawdź: File vs Folder w PathType



Szkolenie przygotowane przez **Robotivity®** — Autoryzowany Partner UiPath
info@robotivity.pl · www.Robotivity.pl · tel.: +48 690 020 576